Adaptive Business Architecture

An English Edition of the ABA Book

This book is about one practical question: why do some software systems feel fast, simple, and flexible in their first year, but become expensive, risky, and slow to change after several years?

The short answer is that many systems do not fail because the programming language was wrong, the database was old, or the framework was unfashionable. They fail because the **cost of change** quietly increases. Ownership becomes unclear. Business boundaries are confused with technical layers. Architectural decisions are forgotten. Each team solves its immediate problem by crossing another team’s boundary. Over time, the system still runs, but changing it becomes painful.

Adaptive Business Architecture, or **ABA**, is a way to design enterprise systems around the long-term cost of change. ABA does not reject Domain-Driven Design, Clean Architecture, Modular Monoliths, Microservices, CQRS, or Event Sourcing. It gives them a higher decision frame: *does this choice make future change safer, cheaper, and more obvious?*

This book uses Identity, Accounting, Payments, Enrollment, and Reporting as teaching examples. They are not presented as universal modules that every product must copy. They are examples that help us see how ABA thinks: how it defines ownership, how it treats contracts as public boundaries, how it separates Platform capabilities from Business decisions, how it turns Governance into executable validation, and how it makes architecture readable by AI agents.

The central sentence of the book is:

> Architecture is not the art of building a system. Architecture is the art of reducing the future cost of change.

How to read this book

Read the book in three passes if you want the best result.

First, read the foundation chapters to understand the philosophy. Focus on cost of change, ownership, context boundaries, and the idea that architecture must be measurable.

Second, read the example chapters. Do not copy them blindly. Instead, ask: in my own system, who owns this decision? Which context owns this data? Which contract is public? Which rule is currently duplicated?

Third, read the implementation and adoption chapters. These chapters explain how to turn ABA into repository structure, metadata, governance rules, CI/CD checks, and AI-agent workflows.

What this book is not

This book is not a framework manual. It does not say that every company must use the same folder structure, the same database, the same message broker, or the same deployment model. ABA is not a replacement for engineering judgment. It is a decision model that makes judgment safer and more explicit.

This book is also not a promise that architecture can remove all complexity. Enterprise systems are complex because businesses are complex. ABA does not eliminate complexity; it tries to put complexity where it belongs and make every unit of complexity justify its cost.

Table of Contents

- Chapter 1: The Birth of ABA
- Chapter 2: The Cost of Change Theory
- Chapter 3: What ABA Is and Is Not
- Chapter 4: Complexity Must Pay Rent
- Chapter 5: Architecture Economics
- Chapter 6: Ownership as the Center of Architecture

- Chapter 7: Business Contexts as Ownership Boundaries
- Chapter 8: Contracts as Public Boundaries
- Chapter 9: Data Ownership and Decision Ownership
- Chapter 10: Conway's Law and Team Ownership
- Chapter 11: ABA and Domain-Driven Design
- Chapter 12: ABA and Clean Architecture
- Chapter 13: ABA and Microservices
- Chapter 14: ABA and Modular Monoliths
- Chapter 15: ABA, CQRS, and Event Sourcing
- Chapter 16: The Platform Layer
- Chapter 17: Messaging, Outbox, and Inbox
- Chapter 18: Security Without Mixing Business Identity
- Chapter 19: Executable Governance
- Chapter 20: Architecture Tests
- Chapter 21: Architecture Telemetry
- Chapter 22: Architecture Decision Records
- Chapter 23: Metadata as Machine-Readable Architecture
- Chapter 24: AI-Navigable Architecture
- Chapter 25: How to Read the Example Domains
- Chapter 26: Identity Example
- Chapter 27: Identity Implementation Walkthrough
- Chapter 28: Accounting Example
- Chapter 29: Accounting Implementation Walkthrough
- Chapter 30: Payments Example
- Chapter 31: Payments Implementation Walkthrough
- Chapter 32: Enrollment Example
- Chapter 33: Enrollment Implementation Walkthrough
- Chapter 34: Reporting Example
- Chapter 35: A Complete Cross-Context Flow
- Chapter 36: Case Study: From Big Ball of Mud to ABA
- Chapter 37: Migration from Layered Architecture

- Chapter 38: Contract Catalog
- Chapter 39: Governance Pipeline in CI/CD
- Chapter 40: Architecture Health Score
- Chapter 41: Repository Template
- Chapter 42: .NET Implementation Reference
- Chapter 43: Anti-Patterns
- Chapter 44: Adoption Guide
- Chapter 45: Practical Review Checklist
- Chapter 46: For Managers and Technical Leaders
- Chapter 47: Final Manifesto
- Appendix A: Glossary
- Appendix B: ownership.yaml Template
- Appendix C: context-map.yaml Template
- Appendix D: ADR Template
- Appendix E: AI Agent Prompt
- Appendix F: Publishing Review Checklist

Chapter 1: The Birth of ABA

> **Chapter motto:** Fast code becomes debt when it is expensive to change.

The problem that created ABA

Imagine a product team in its first year. The codebase is small, the team sits together, and new features move quickly. A developer can add a field, change a table, adjust an API, and ship by the end of the week. Everyone feels that the architecture is working.

Now imagine the same system four years later. The product is successful, but every change is now surrounded by fear. A simple change in registration touches Identity, Payments, Reporting, and Accounting. No one is completely

sure who owns the rule. A developer asks, "Can I update this table?" Another answers, "I think so, but check with the finance team." A third person says, "That table is also used by a scheduled job." Suddenly the change is not a small task. It is a coordination problem.

ABA was born from this kind of pain. The pain is not simply technical. It is economic and organizational. The system has become expensive to change.

The wrong first question

Many architecture discussions begin with a familiar debate: should we build a monolith or microservices? Should we use Clean Architecture? Should we use Event Sourcing? Should every context have its own database?

Those questions matter, but they are not the first question. The first question should be:

Will this decision reduce the future cost of change, or will it only make today's design look sophisticated?

When teams skip this question, they often adopt patterns before they understand the reason. They split services before ownership is clear. They add layers before business boundaries are clear. They introduce event streams before the meaning of events is stable.

The ABA answer

ABA treats architecture as a system for keeping future change safe, cheap, and obvious. It cares less about whether a diagram looks impressive and more about whether a developer can answer these questions:

- Who owns this data?
- Who owns this business rule?
- Which context can change this decision?
- Which contract is safe for other contexts to depend on?
- How will we know if the architecture is getting worse?

If these questions are hard to answer, the architecture is already drifting.

A simple example

A payment is captured. Should Payments create the accounting journal directly? At first, this seems efficient. Payments already knows the amount. It can write a journal row and move on. But ABA asks a different question: who owns financial truth?

The answer is Accounting. Payments owns money movement; Accounting owns financial truth. Therefore Payments should publish a contract such as `PaymentCapturedV1`. Accounting consumes that contract and decides how to post the journal.

This looks slightly slower on day one. It is cheaper on year five.

Chapter checklist

- Do we measure architecture by beauty, or by future change cost?
- Can a new engineer identify ownership without asking five people?
- Are we using patterns because they reduce change cost, or because they sound modern?

Chapter 2: The Cost of Change Theory

> **Chapter motto:** The real price of software is paid when the business changes.

What cost of change really means

The cost of change is not just coding time. It includes analysis, coordination, meetings, pull-request reviews, test failures, migration work, deployment risk, user communication, rollback planning, and incident recovery.

A change may take two hours to code and two weeks to safely release. ABA cares about the two weeks.

A useful mental formula

A simple way to think about architectural value is:

```
Architectural Value = Future Change Cost Reduced - Complexity Added Today
```

This formula is not meant to be mathematically exact. It is meant to change the conversation. A new pattern, framework, service, layer, or metadata file is not free. It adds complexity. The only reason to accept that complexity is that it reduces a larger future cost.

Why popular patterns sometimes fail

A team may introduce CQRS because it is popular. If the write model and read model are truly different, CQRS can reduce cost. But if the feature is a simple CRUD screen, CQRS may only add handlers, projections, eventual consistency, and debugging complexity.

A team may introduce microservices because the company expects scale. If teams need independent ownership, deployment, and scaling, microservices may be justified. But if the same team changes all services together, the organization now pays distributed-system costs without gaining independence.

A team may use Event Sourcing because auditability matters. For a trading platform, banking ledger, or business process with rich history, it can be valuable. For a simple profile editor, it may be overkill.

ABA does not reject these tools. It asks each tool to pay rent.

Decision rule

Before adopting any architectural mechanism, answer three questions:

1. What future change will this make easier?
2. What present complexity will it add?
3. What evidence tells us that the tradeoff is worth it?

If the team cannot answer these questions, the decision should be delayed, simplified, or recorded as an experiment.

Example: adding a notification system

A direct implementation sends emails from each business feature. It is fast at first. Enrollment sends an email after activation. Payments sends an email after refund. Identity sends an email after invitation.

After a year, templates are duplicated, retry behavior differs, unsubscribe logic is inconsistent, and audit logs are incomplete.

ABA would move the technical capability of delivery to Platform.Notifications while keeping business decisions in each context. Enrollment decides *that* an enrollment activation notification is needed. Platform.Notifications handles delivery mechanics, templates, retries, provider failures, and observability.

The architecture adds a Platform module, but it reduces repeated future change.

Chapter 3: What ABA Is and Is Not

> **Chapter motto:** A method is useful only when it says no to the wrong things.

What ABA is

ABA is an architectural decision model for long-lived enterprise systems. It organizes architecture around business ownership, future change cost, executable governance, observability, and AI readability.

ABA is not a single pattern. It is closer to an operating model for architecture. It helps a team decide when to use patterns such as Clean Architecture, DDD, Modular Monolith, Microservices, CQRS, and Event Sourcing.

What ABA is not

ABA is not a framework, a NuGet package, a folder template, or a magic diagram. A repository can follow the folder structure of ABA and still violate ABA if ownership is unclear. Another repository can use different names and still follow ABA if ownership, contracts, governance, telemetry, and intent are explicit.

ABA is also not anti-microservice or anti-monolith. It is against premature distribution and against accidental coupling. A modular monolith with clear ownership is often better than many services with shared confusion. But a service split with strong ownership and evidence can be perfectly aligned with ABA.

The core promise

ABA promises a practical target:

Make change safe, cheap, and obvious.

Safe means that changes do not silently corrupt data or violate ownership. Cheap means that changes do not require unnecessary coordination. Obvious means that developers and AI agents can quickly understand where a change belongs.

How ABA relates to engineering culture

ABA only works when teams accept ownership as a real discipline. If every team can change every table, no architecture document can save the system. If deadlines always justify crossing boundaries, governance becomes theater. If decisions are not recorded, future teams repeat the same debates.

ABA is therefore both technical and cultural. It asks the organization to treat architecture as a long-term asset.

Chapter 4: Complexity Must Pay Rent

> **Chapter motto:** Every abstraction should explain the future cost it reduces.

The problem of prestigious complexity

Some complexity enters a system because the business requires it. Some enters because engineers like elegant designs. ABA is not against elegance, but it is suspicious of complexity that cannot explain its economic purpose.

Prestigious complexity is especially dangerous. It arrives with names that sound professional: distributed saga, event-sourced aggregate, generic repository, universal workflow engine, dynamic rule platform. Sometimes these tools are necessary. Sometimes they are monuments to uncertainty.

Rent questions

When a team proposes complexity, ask:

- What specific future change becomes cheaper?
- What specific risk becomes lower?
- What specific ownership boundary becomes clearer?
- How will we remove it if the assumption proves false?

If the answer is vague, the complexity is not paying rent.

Example: a generic rule engine

A product team expects pricing rules to change often. Someone proposes a generic rule engine. The proposal may be valid. But ABA asks for evidence. Are rules truly business-configurable? Who owns rule changes? Can non-developers safely change them? How will rules be tested? How will auditability work? What happens when a rule requires data from Accounting or Enrollment?

If these answers are not clear, a small explicit domain model may be better than a generic engine.

Practical guidance

Start with the simplest design that preserves ownership and makes future evolution possible. Add complexity when the pressure is visible. Complexity added before evidence often becomes architecture debt.

Chapter 5: Architecture Economics

> **Chapter motto:** Architecture is a financial decision disguised as a technical one.

Architecture as investment

Every architectural decision invests engineering time today in exchange for expected flexibility tomorrow. Some investments pay back quickly. Others never do.

A separate Contract project may feel like overhead in the first week. It pays back when other contexts can depend on stable public models without referencing internal domain code. A governance test may feel strict today. It pays back when it prevents a dependency that would have taken months to remove later.

The hidden interest rate

Architecture debt has interest. A direct database read across context boundaries may save one day now. If it becomes widely used, removing it later may require data migration, contract design, consumer coordination, and release sequencing.

The interest rate of architectural debt is high because it compounds through usage. The more people depend on a wrong shortcut, the more expensive it becomes.

Enterprise systems need economic language

Business leaders understand cost, risk, and time-to-market better than they understand abstract purity. ABA gives architects a way to explain design decisions in business terms.

Instead of saying, "This violates clean architecture," say:

"This dependency makes future changes to Accounting require changes in Payments. It increases coordination cost and reduces independent release ability."

That explanation is harder to dismiss.

Chapter 6: Ownership as the Center of Architecture

> **Chapter motto:** When ownership is unclear, every change becomes a negotiation.

The central idea

Ownership is the strongest concept in ABA. Every important data element, decision, rule, and contract must have exactly one owner. Not zero. Not two. Exactly one.

This does not mean only one developer can edit the code. It means one context has the authority to decide the meaning and lifecycle of that concept.

Four ownership types

ABA separates ownership into four useful categories.

Data ownership asks: who owns this data as a source of truth?

Rule ownership asks: who owns the business invariant?

Decision ownership asks: who has final authority over behavior?

Contract ownership asks: who publishes and evolves the public contract?

A system may have clear data ownership but unclear rule ownership. For example, Identity may own `User`, but Enrollment may own the rule that a participant cannot enroll twice in the same offering. These rules must not be confused.

Example: User and Enrollment

Identity owns the user. Enrollment owns the enrollment relationship. If Enrollment needs to know that a user exists, it should use an Identity contract or query, not directly modify the User table.

The distinction sounds simple, but it prevents many failures. Without it, Enrollment may start storing role data, Payments may store user status, and Reporting may become a hidden source of truth.

Ownership metadata

ABA records ownership in machine-readable files.

```
context: Identity

data_ownership:
  User:
    owner: Identity
  Role:
    owner: Identity
  Permission:
    owner: Identity
  Membership:
    owner: Identity
```

```
forbidden_writers:
  - Payments
  - Enrollment
  - Reporting
```

This file is not decoration. It should be read by governance tools and AI agents. If ownership is only written in a wiki, it will be forgotten.

Chapter 7: Business Contexts as Ownership Boundaries

> **Chapter motto:** A context is not a folder; it is a boundary of authority.

Context is often misunderstood

Many teams use the word context when they really mean module, service, namespace, or project. ABA uses context in a stricter way: a context is a business ownership boundary.

A context is valid when it can answer the question: what business capability do you own?

Identity owns identity and access concepts. Accounting owns financial truth. Payments owns payment execution. Enrollment owns participation. Reporting owns visibility and analysis.

Context vs service

A context may be implemented inside a modular monolith. It may later become a service. It may share a deployment unit with other contexts. The implementation form can change. The ownership boundary should be stable.

This is where ABA differs from many microservice discussions. A service boundary without ownership is just distributed confusion.

Example of a weak boundary

A system has services named `UserService`, `PaymentService`, `ReportService`, and `AdminService`. AdminService updates user status, payment flags, and reporting categories because the admin screen needs everything. At first, this looks practical. Later, AdminService becomes the most dangerous service in the system because it crosses every boundary.

ABA would ask: what decisions does Admin own? Usually none. Admin is a user interface or workflow layer. It should orchestrate through contracts, not own other contexts' data.

Context structure

A context can be organized like this:

```
Identity/
  Contracts/
  Domain/
  Application/
  Infrastructure/
  _Docs/
```

The structure is not the architecture by itself. The architecture is the rule that only `Contracts` is public to other contexts.

Chapter 8: Contracts as Public Boundaries

> **Chapter motto:** A contract is the public promise of a context.

Why contracts matter

If contexts communicate through internal domain objects, they are not independent. They are only separated by folders. A contract is the explicit public surface that other contexts can depend on.

Contracts include integration events, commands, queries, DTOs, response models, and published schemas. They are not internal entities.

Entity is not contract

A `User` entity may contain internal state, validation behavior, persistence assumptions, and lifecycle rules. A `UserCreatedV1` event is a public statement that a user was created. These are not the same thing.

Sharing entities across contexts creates shared domain models. ABA treats shared domain models as a major anti-pattern.

Versioning

Contracts should be versioned.

```
public sealed record UserCreatedV1(
    Guid UserId,
    string Email,
    DateTime CreatedAt
);
```

If a breaking change is needed, create `UserCreatedV2`. Do not silently change `UserCreatedV1`. Consumers must be able to evolve safely.

Contract ownership

A contract has one owner. `PaymentCapturedV1` belongs to Payments. `JournalPostedV1` belongs to Accounting. Reporting may consume both, but it does not own either.

This rule prevents contract chaos. Without ownership, consumers pressure contracts to fit their local needs, and the public surface becomes unstable.

Chapter 9: Data Ownership and Decision Ownership

> **Chapter motto:** The table owner and the decision owner are not always the same thing.

Why the distinction matters

Many systems define ownership only at the database level. This is not enough. A table may store data, but the business decision about how to use that data may belong elsewhere.

For example, Identity owns users. Enrollment owns whether a user can enroll in a particular offering. Payments owns whether a payment was captured. Accounting owns how captured payments become journal entries.

Decision ownership example

A user wants to enroll in a course that requires payment. Identity can confirm the user exists. Payments can confirm payment status. Enrollment owns the decision to activate the enrollment. Accounting owns the decision to post the financial journal.

If Payments activates Enrollment directly, the decision boundary is broken. If Enrollment creates a Journal directly, financial ownership is broken.

The cost of mixing ownership

When data ownership and decision ownership are mixed, changes become unpredictable. A payment rule change might break enrollment. A membership change might affect accounting. The team loses the ability to reason locally.

ABA does not require every context to be isolated from every other context. It requires that the direction of authority be explicit.

Chapter 10: Conway's Law and Team Ownership

> **Chapter motto:** Your architecture will eventually look like your communication paths.

The organizational side

Architecture is not only code. It is also the shape of the organization. If five teams can change the same concept, the code will eventually reflect that confusion.

ABA aligns context ownership with team responsibility where possible. A team should own a coherent business capability, not a random slice of technical layers.

Layer teams vs capability teams

A team that owns all controllers but no domain decisions becomes a delivery bottleneck. A team that owns all database changes becomes a coordination bottleneck. A team that owns a business context can make decisions with less coordination.

This does not mean every context needs a dedicated team from day one. It means the architecture should be ready for that evolution.

Split only when ownership exists

Splitting a context into a separate service before a team can own it independently often creates operational cost without organizational benefit. ABA prefers ownership clarity before technical separation.

Chapter 11: ABA and Domain-Driven Design

> **Chapter motto:** DDD gives language and models; ABA adds ownership, governance, and change economics.

Where ABA agrees with DDD

ABA is strongly aligned with several DDD ideas. It values bounded contexts, ubiquitous language, domain modeling, aggregates, and business meaning. ABA also agrees that technical layers should not define the business.

Where ABA extends the conversation

DDD helps a team model the domain. ABA asks how that model will remain safe to change over many years. ABA makes ownership explicit in metadata. It asks governance tools to enforce boundaries. It measures architecture telemetry. It also considers how AI agents will understand the architecture.

DDD may say that Identity and Accounting are different bounded contexts. ABA asks:

- Where is that ownership recorded?
- What contract is public?
- What test prevents boundary violations?
- What metric shows whether coupling is increasing?
- How will an AI agent know not to cross the boundary?

Practical example

DDD may model `Journal` as an aggregate in Accounting. ABA agrees. Then ABA adds that `Journal` ownership must appear in `ownership.yaml`, public events must be versioned, and governance must prevent other contexts from referencing `Accounting.Domain`.

ABA is not a replacement for DDD. It is a governance and evolution layer around domain thinking.

Chapter 12: ABA and Clean Architecture

> **Chapter motto:** Clean layers protect code direction; ABA protects business ownership.

What Clean Architecture gives us

Clean Architecture is valuable because it protects the domain from infrastructure concerns. It encourages dependencies to point inward. It helps keep business rules independent from frameworks.

ABA supports this idea inside each context. A context can have Domain, Application, Infrastructure, and Contracts projects. Domain should not depend on EF Core, ASP.NET, or message brokers.

What Clean Architecture does not fully answer

Clean Architecture does not automatically tell us which business context owns a rule. It can produce a cleanly layered system where every module still shares the same domain model.

A solution can have perfect layers and still have unclear ownership.

ABA's addition

ABA uses Clean Architecture as an internal structure, not as the whole architecture. Clean layers answer how code dependencies should flow inside a context. ABA answers who owns the business capability and how contexts evolve together.

Example

A `PaymentService` inside a clean application layer may still call `AccountingRepository` directly. The code may look layered, but ownership is broken. ABA would require Payments to publish `PaymentCapturedV1` and Accounting to decide how to post the journal.

Chapter 13: ABA and Microservices

> **Chapter motto:** A microservice without ownership is a remote procedure call with deployment pain.

The promise of microservices

Microservices promise independent deployment, independent scaling, fault isolation, and team autonomy. These are real benefits when the organization is ready for them.

The failure mode

Many teams split services around technical nouns or database tables rather than ownership. They create `UserService`, `AdminService`, `ReportService`, and `CommonService`. Then every feature crosses multiple services, and the system becomes slower to change than the monolith it replaced.

ABA's rule

Do not split because a diagram looks modern. Split when ownership, deployment pressure, scaling pressure, or regulatory pressure justifies the cost.

Modular monolith first

ABA often recommends a modular monolith as the first implementation form. This does not mean ignoring boundaries. It means enforcing boundaries inside one deployable unit until evidence suggests a context should be extracted.

A modular monolith with strong governance can be a better starting point than premature microservices.

Chapter 14: ABA and Modular Monoliths

> **Chapter motto:** One deployment can contain many ownership boundaries.

Why modular monoliths fit ABA

A modular monolith allows teams to keep operational complexity low while still modeling business boundaries. It is especially useful when the product is growing but ownership is not mature enough for distributed services.

The danger

A monolith becomes dangerous when modules are only folders and all code can reference all code. ABA requires module boundaries to be enforced by tests, analyzers, and metadata.

Evolution path

Start with contexts inside one repository and one deployment. Enforce contracts and dependencies. Measure cross-context changes. If a context gains independent team ownership, scaling needs, or release pressure, extract it with evidence.

Example extraction trigger

Payments may start inside the modular monolith. Later, payment provider reliability, compliance requirements, and independent release cycles may justify extracting Payments into a service. Because contracts were already explicit, extraction becomes less risky.

Chapter 15: ABA, CQRS, and Event Sourcing

> **Chapter motto:** History is valuable, but not every feature deserves a time machine.

CQRS

CQRS separates write models from read models. ABA supports CQRS when reads and writes have different shapes, performance needs, or ownership concerns. Reporting is a natural candidate because it consumes events and builds read models.

ABA does not require CQRS everywhere. A simple reference-data screen may not need separate models.

Event Sourcing

Event Sourcing stores changes as a sequence of events. It can be powerful when history is the source of truth. But it also adds cognitive, operational, and debugging complexity.

Business immutability first

For Accounting, ABA emphasizes business immutability: posted journals should not be edited or deleted. This can be implemented through append-only

journals and reversal entries without necessarily adopting a full Event Sourcing framework.

The architectural question is not "Can we use Event Sourcing?" The question is "What level of historical integrity does the business need, and what is the cheapest safe way to provide it?"

Chapter 16: The Platform Layer

> **Chapter motto:** Platform provides capabilities; business contexts own decisions.

Why Platform exists

Enterprise systems repeatedly need technical capabilities: messaging, persistence, caching, security integration, observability, background jobs, file storage, and notifications. If every business context implements these capabilities differently, the system becomes inconsistent and expensive to operate.

Platform centralizes reusable technical capabilities.

What Platform must not do

Platform must not own business decisions. It should not decide whether a journal can be posted, whether a refund is allowed, or whether an enrollment can be activated. Those decisions belong to business contexts.

Example: messaging

Platform.Messaging owns outbox storage, serialization, retries, dead-letter behavior, correlation IDs, and broker integration. Accounting owns the decision to publish `JournalPostedV1`. Payments owns the decision to publish `PaymentCapturedV1`.

Example: security

Platform.Security validates tokens, resolves current user information, and provides current tenant context. Identity owns users, roles, permissions, and memberships. This separation prevents technical security plumbing from becoming a business identity module.

Chapter 17: Messaging, Outbox, and Inbox

> **Chapter motto:** A message that can be lost is not an integration contract; it is a hope.

Why messaging matters

When contexts communicate through events, reliability becomes critical. If Payments captures money but the event is lost before Accounting sees it, financial truth becomes inconsistent.

ABA therefore treats reliable messaging as a Platform capability.

Outbox pattern

The outbox pattern stores the business change and the message in the same database transaction. A background process later publishes the message to the broker.

```
1. Payment is captured
2. Payment state is saved
3. PaymentCapturedV1 is written to Outbox
4. Transaction commits
5. Publisher sends Outbox message
6. Accounting consumes the event
```

Inbox pattern

The inbox pattern prevents duplicate processing. If a message is delivered twice, the consumer can detect that it has already processed the message.

Ownership reminder

The Platform owns delivery mechanics. Business contexts own event meaning.

Chapter 18: Security Without Mixing Business Identity

> **Chapter motto:** Authentication is a protocol; identity is a business capability.

The common mistake

Many systems put users, roles, token validation, OAuth configuration, permission logic, and organization membership into one security module. The result is a module that is both technical and business-critical, and therefore hard to change.

ABA separates Identity from Platform.Security.

Platform.Security owns

- JWT validation
- OIDC/OAuth integration
- current user abstraction
- current tenant resolution
- policy execution mechanics
- secure configuration patterns

Identity owns

- user lifecycle
- role lifecycle
- permission definitions
- organization membership
- invitations
- authorization policies as business concepts

Why this matters

If a token provider changes, Identity should not be rewritten. If a permission rule changes, token validation should not change. Separating these concerns keeps changes smaller and safer.

Chapter 19: Executable Governance

> **Chapter motto:** A rule that cannot fail a build is only a suggestion.

Why documentation is not enough

Architecture rules written in a wiki are useful, but they are not sufficient. People forget. Teams rush. AI agents optimize for local completion. Code reviews miss details.

Executable governance turns architecture rules into tests, analyzers, and pipeline checks.

Examples of executable rules

- `Accounting.Domain` must not reference `Payments.Domain`.
- Contracts must be versioned.

- Every context must contain ownership metadata.
- Shared domain models are forbidden.
- Posted journals cannot be updated by application commands.

Build failure as architecture protection

Failing a build should not be seen as bureaucracy. It is cheaper than discovering a broken boundary six months later.

The best governance is quiet. It runs automatically. It catches violations early. It explains the reason clearly.

Chapter 20: Architecture Tests

> **Chapter motto:** Test the architecture before the architecture tests you.

What architecture tests verify

Architecture tests verify structural rules: dependencies, naming, project boundaries, metadata presence, and contract versioning. They are not unit tests for business behavior. They are tests for architectural integrity.

Example rule

```
[Fact]
public void AccountingDomain_ShouldNotReference_PaymentsDomain()
{
    var result = Types.InAssembly(AccountingDomainAssembly)
        .ShouldNot()
        .HaveDependencyOn("Payments.Domain")
        .GetResult();

    Assert.True(result.IsSuccessful);
}
```

Why these tests matter

Without this test, one convenient reference may enter the codebase. After that, other developers copy the pattern. Eventually the dependency becomes normal. Removing it becomes expensive.

Architecture tests prevent small shortcuts from becoming permanent structure.

Chapter 21: Architecture Telemetry

> **Chapter motto:** You cannot improve an architecture you cannot see.

Operational telemetry vs architecture telemetry

Operational telemetry tells us whether the system is running. Architecture telemetry tells us whether the system is becoming harder to change.

Both are necessary. A system can have excellent CPU metrics and terrible architectural health.

Important metrics

ABA recommends tracking:

- lead time for change
- deployment frequency
- change failure rate
- mean time to recovery
- cross-context change rate
- ownership violations
- dependency growth
- contract volatility

- governance failure trends

Example interpretation

If cross-context changes increase every month, context boundaries may be wrong or unclear. If contract volatility is high, public models may not be stable. If ownership violations appear repeatedly in the same area, team boundaries and metadata may need review.

Architecture health score

A health score is not a perfect truth. It is a conversation starter. It should combine dependency health, ownership clarity, contract stability, governance results, and change flow metrics.

Chapter 22: Architecture Decision Records

> **Chapter motto:** Future teams inherit decisions, not intentions.

Why ADRs exist

Code shows what exists. It rarely explains why it exists. Future teams need the reason behind important decisions.

An Architecture Decision Record, or ADR, captures the context, decision, alternatives, tradeoffs, and consequences.

Example

```
# ADR-004: Posted Journals Are Immutable

## Decision
Posted journals cannot be updated or deleted.
```

```
## Reason
Financial auditability requires a trustworthy history.

## Alternatives
Allow updates with audit logs.
Use full Event Sourcing.

## Consequences
Corrections must use reversal journals.
```

When to create an ADR

Create an ADR when a decision changes ownership, introduces infrastructure, creates a context, changes a contract strategy, splits a service, adopts a major framework, or intentionally accepts a tradeoff.

ADRs preserve intent. Without intent, future refactoring becomes guesswork.

Chapter 23: Metadata as Machine-Readable Architecture

> **Chapter motto:** If AI cannot read the architecture, it will invent one.

Why metadata matters

Human developers can ask teammates for context. AI agents cannot reliably infer architectural intent unless the repository exposes it. ABA therefore treats metadata as part of the architecture.

Required metadata

A context should include:

- `ownership.yaml`
- `context-map.yaml`
- `decisions.md`

- `evolution.md`
- `ai-rules.yaml`
- `contract-catalog.yaml`

Example

```
context: Payments

publishes:
  - PaymentCapturedV1
  - PaymentRefundedV1

consumes:
  - UserCreatedV1

allowed_dependencies:
  - Identity.Contracts
  - Platform.Messaging

forbidden_dependencies:
  - Accounting.Domain
  - Enrollment.Domain
```

Metadata is not a replacement for good code. It is a navigation layer for humans, tools, governance, and AI agents.

Chapter 24: AI-Navigable Architecture

> **Chapter motto:** AI should accelerate good architecture, not automate architectural drift.

The new problem

AI agents can write code quickly. This is useful, but dangerous. If the agent does not understand ownership boundaries, it can create coupling faster than humans can review it.

ABA's rule for AI

Before generating code, an AI agent must read the architecture.

The agent should read documentation, metadata, ADRs, context maps, and AI rules before changing code.

Pull request workflow

A safe AI workflow looks like this:

```
1. Read repository architecture documents
2. Identify the target business context
3. Check ownership metadata
4. Check allowed dependencies
5. Design contract changes if needed
6. Generate code
7. Generate tests
8. Update metadata
9. Run governance checks
10. Explain the architectural impact
```

Failure mode

An agent asked to "activate enrollment after payment" may directly update Enrollment from Payments. This solves the local task but violates ownership. The correct design is for Payments to publish `PaymentCapturedV1` and Enrollment to consume it according to its own activation policy.

Chapter 25: How to Read the Example Domains

> **Chapter motto:** Examples are not templates to copy; they are decisions to study.

Why this book uses example domains

The book uses Identity, Accounting, Payments, Enrollment, and Reporting because they expose different architectural pressures.

Identity shows ownership of users, roles, permissions, and membership. Accounting shows immutability and financial truth. Payments shows provider integration and idempotency. Enrollment shows policy, capacity, and workflow. Reporting shows read models without source-of-truth ownership.

What not to do

Do not copy these domains blindly into every system. A small product may not need a full Accounting context. A marketplace may need additional contexts such as Catalog, Pricing, Billing, Taxation, or Fulfillment.

What to learn

Use the examples to learn how ABA asks questions:

- Who owns the data?
- Who owns the decision?
- What is the public contract?
- What belongs to Platform?
- What must governance enforce?
- What telemetry indicates architectural drift?

Chapter 26: Identity Example

> **Chapter motto:** Identity owns who the actor is; Platform owns how credentials are verified.

Scope

Identity owns digital identity concepts: User, Role, Permission, Organization, Membership, Invitation, and authorization policy as a business concept.

It does not own token validation mechanics. That belongs to Platform.Security.

Common failure

A system starts with a shared `Users` table. Payments reads it. Enrollment writes status fields. Reporting joins it directly. Admin screens update roles without going through Identity. Over time, no one knows whether a user is active because Identity says so, because Payments says so, or because a flag in Reporting was updated.

ABA design

Identity becomes the owner of user lifecycle and membership. Other contexts consume contracts.

```
context: Identity

data_ownership:
  User:
    owner: Identity
  Role:
    owner: Identity
  Permission:
    owner: Identity
  Membership:
    owner: Identity

published_contracts:
  - UserCreatedV1
```

```
  - UserDisabledV1
  - RoleAssignedV1
  - MembershipActivatedV1
```

Implementation notes

The User aggregate should protect lifecycle transitions. A disabled user cannot authenticate. A suspended membership cannot grant access. Role assignment should be audited.

Identity publishes events. It does not allow other contexts to write its tables.

Chapter 27: Identity Implementation Walkthrough

> **Chapter motto:** A user table is not an identity architecture.

Aggregate design

A simple User aggregate may expose methods such as `Activate`, `Suspend`, `Disable`, and `ChangeEmail`. These methods enforce rules and produce domain events.

```
public sealed class User
{
    public Guid Id { get; private set; }
    public string Email { get; private set; }
    public UserStatus Status { get; private set; }

    public void Disable(string reason)
    {
        if (Status == UserStatus.Deleted)
            throw new DomainException("Deleted users cannot be disabled.");

        Status = UserStatus.Disabled;
        AddDomainEvent(new UserDisabled(Id, reason));
```

```
    }
}
```

Application layer

The application layer orchestrates use cases. It loads the aggregate, calls domain behavior, saves changes, and relies on Platform.Messaging to publish integration events.

Governance rule

No context may reference `Identity.Domain`. Other contexts may reference `Identity.Contracts` only.

Why this matters

If Enrollment needs to know whether a user exists, it should use a contract or application API. It should not validate Identity rules by reading Identity tables directly.

Chapter 28: Accounting Example

> **Chapter motto:** Financial history should be corrected, not rewritten.

Scope

Accounting owns financial truth. It owns accounts, journals, journal lines, ledger entries, fiscal periods, posting rules, trial balances, and financial corrections.

Core rule

Posted journals are immutable. If a journal is wrong, create a reversal or correction. Do not update or delete the historical record.

Why ABA treats this differently

A generic CRUD approach would add `UpdateJournal` and `DeleteJournal` because users ask for correction. ABA asks what the business requires. Financial auditability requires trustworthy history. Therefore the design must make unsafe operations impossible.

Contract example

```
public sealed record JournalPostedV1(
    Guid JournalId,
    string ReferenceNumber,
    DateTime PostingDate,
    decimal TotalDebit,
    decimal TotalCredit,
    DateTime PublishedAt
);
```

Comparison with Event Sourcing

A full Event Sourcing system can preserve history, but ABA does not require it by default. Accounting can often achieve business immutability through append-only journals, ledger entries, and reversal entries. Use Event Sourcing when its extra complexity pays rent.

Chapter 29: Accounting Implementation Walkthrough

> **Chapter motto:** The ledger is a memory system, not an editable spreadsheet.

Journal aggregate

A Journal contains lines. Each line has an account, debit or credit side, and amount. The aggregate validates that total debit equals total credit before posting.

```
public void Post(FiscalPeriod period)
{
    if (!period.IsOpen)
        throw new DomainException("Cannot post into a closed fiscal period.");

    if (TotalDebit != TotalCredit)
        throw new DomainException("Debit and credit must balance.");

    if (Status != JournalStatus.Validated)
        throw new DomainException("Only validated journals can be posted.");

    Status = JournalStatus.Posted;
    PostedAt = DateTime.UtcNow;
    AddDomainEvent(new JournalPosted(Id));
}
```

Forbidden commands

Commands such as `UpdatePostedJournal` or `DeletePostedJournal` should not exist. Governance can search for these patterns and fail the build if they appear.

Integration with Payments

Accounting consumes `PaymentCapturedV1`. Payments does not create journals. This preserves financial ownership.

Chapter 30: Payments Example

> **Chapter motto:** Payment execution is not the same as financial truth.

Scope

Payments owns money movement: payment attempts, authorization, capture, refund, settlement tracking, and provider integration.

It does not own financial posting. Accounting owns that.

Common failure

A payment provider callback arrives. The Payments code updates the payment, creates an accounting journal, activates enrollment, and sends an email. The local workflow works, but ownership is destroyed. Payments becomes a god context.

ABA design

Payments records the payment state and publishes events. Other contexts react according to their own ownership.

```
Provider callback -> Payments updates Payment
Payments publishes PaymentCapturedV1
Accounting posts journal
Enrollment activates enrollment if its policy allows
Reporting updates read model
```

Idempotency

Provider callbacks may be repeated. Payments must treat repeated callbacks safely. The same capture should not create multiple state transitions or multiple events.

Chapter 31: Payments Implementation Walkthrough

> **Chapter motto:** A provider response is input; payment state is the owned truth.

Payment aggregate

The Payment aggregate protects lifecycle transitions. A captured payment cannot be captured twice. A failed payment cannot be settled. A refund cannot exceed the captured amount.

```
public void Capture(string providerTransactionId, Money amount)
{
    if (Status == PaymentStatus.Captured)
        return; // idempotent handling

    if (Status != PaymentStatus.Authorized)
        throw new DomainException("Only authorized payments can be
captured.");

    CapturedAmount = amount;
    ProviderTransactionId = providerTransactionId;
    Status = PaymentStatus.Captured;
    AddDomainEvent(new PaymentCaptured(Id, amount));
}
```

Provider adapters

Provider-specific logic belongs in Infrastructure. The domain should not depend on Stripe, PayPal, Adyen, or any provider SDK.

Outbox requirement

A captured payment without a published event creates system inconsistency. Payments should use the Platform Outbox.

Chapter 32: Enrollment Example

> **Chapter motto:** Enrollment owns participation, not identity and not money movement.

Scope

Enrollment owns participation in an offering. It manages enrollment requests, approval, capacity, waitlists, activation, cancellation, completion, and enrollment policies.

Common failure

A system treats enrollment as a row that any module can update. Payments activates it. Admin cancels it directly. Reporting adds flags. Capacity is checked in multiple places. Eventually no one knows which rule is authoritative.

ABA design

Enrollment owns enrollment state transitions. Payments may publish a payment event, but Enrollment decides whether payment completion is enough to activate the enrollment.

Example states

```
Requested -> PendingApproval -> PendingPayment -> Enrolled -> Completed
Requested -> Waitlisted
Enrolled -> Cancelled
```

Capacity

Capacity is not a generic technical constraint. It is a business rule. Enrollment owns it.

Chapter 33: Enrollment Implementation Walkthrough

> **Chapter motto:** A seat is not free until the enrollment policy says it is free.

Enrollment policy

An offering may require approval, payment, capacity, prerequisites, or age validation. ABA keeps these rules in Enrollment unless another context clearly owns the rule.

Payment integration

When Enrollment receives `PaymentCapturedV1`, it does not blindly activate every record. It checks policy.

```
public void MarkPaymentReceived(Guid paymentId)
{
    PaymentId = paymentId;

    if (Policy.RequiresApproval && !Approved)
    {
        Status = EnrollmentStatus.PendingApproval;
        return;
    }

    if (!Offering.HasCapacity())
    {
        Status = EnrollmentStatus.Waitlisted;
        return;
    }

    Status = EnrollmentStatus.Enrolled;
    AddDomainEvent(new EnrollmentActivated(Id));
}
```

Why this matters

The payment event is evidence. The enrollment decision belongs to Enrollment.

Chapter 34: Reporting Example

> **Chapter motto:** Reporting creates visibility; it does not become the source of truth.

Scope

Reporting owns dashboards, report definitions, KPIs, analytics models, and read models. It does not own the original business facts.

Read models

Reporting can denormalize data. It can store a model that joins user, payment, enrollment, and journal information. This is allowed because reporting is optimized for reading.

The rule is that Reporting must not become the place where business truth is edited.

Common failure

A report needs a correction, so someone edits the reporting table. Later the source system publishes a new event and overwrites the value. No one knows which value is correct.

ABA says: correct the source context. Let Reporting rebuild or adjust its projection.

Chapter 35: A Complete Cross-Context Flow

> **Chapter motto:** One business process can cross contexts without sharing ownership.

Scenario

A student enrolls in a paid course. The workflow touches Identity, Enrollment, Payments, Accounting, Reporting, and Notifications.

ABA flow

```
1. Identity confirms the user and membership.
2. Enrollment creates an enrollment request.
3. Payments creates and captures payment.
4. Payments publishes PaymentCapturedV1.
5. Enrollment consumes the event and applies its policy.
6. Enrollment publishes EnrollmentActivatedV1.
7. Accounting consumes PaymentCapturedV1 and posts the journal.
8. Reporting consumes public events and updates read models.
9. Platform.Notifications sends messages based on business events.
```

What did not happen

Payments did not create a journal. Accounting did not activate enrollment. Reporting did not update source tables. Platform did not decide business policy.

This is the practical meaning of ABA.

Chapter 36: Case Study: From Big Ball of Mud to ABA

> **Chapter motto:** A system is not rescued by a new framework; it is rescued by clearer ownership.

Starting point

The company has a successful product. Over time, the codebase has accumulated shortcuts:

- A shared `Users` table is read and written by many modules.
- Payments directly creates accounting rows.
- Reporting reads production tables from every module.
- Admin screens bypass domain logic.
- No ADRs explain why decisions were made.
- AI-generated pull requests introduce direct dependencies.

Step 1: map ownership

The team lists important concepts and assigns owners. User belongs to Identity. Journal belongs to Accounting. Payment belongs to Payments. Enrollment belongs to Enrollment. Dashboard belongs to Reporting.

Step 2: protect contracts

The team defines public events and removes direct domain references. Consumers depend on contracts, not internal code.

Step 3: add governance

Architecture tests fail builds when modules reference forbidden namespaces. Metadata checks verify that contexts have ownership files.

Step 4: migrate gradually

The team does not rewrite everything. It starts with one flow: paid enrollment. It introduces `PaymentCapturedV1`, `EnrollmentActivatedV1`, and `JournalPostedV1`. Then it moves direct writes behind context-owned handlers.

Result

The system is still not perfect. But change becomes more predictable. The team now knows where changes belong.

Chapter 37: Migration from Layered Architecture

> **Chapter motto:** Layers organize code; ownership organizes change.

Starting structure

Many systems begin with this structure:

```
Controllers/
Services/
Repositories/
Entities/
Infrastructure/
```

This is not wrong, but it is not enough. It organizes by technical role, not by business ownership.

Migration approach

Do not rewrite the system immediately. Start by identifying business capabilities inside the existing layers. Group code by ownership. Introduce context folders gradually.

Practical sequence

1. Identify the most painful change area.
2. Map data, rules, and decisions.
3. Define a target context.
4. Move contracts first.
5. Move domain rules second.
6. Add governance tests.
7. Migrate consumers gradually.

Example

If payment and accounting logic are mixed in a service called `RegistrationService`, do not simply rename it. Split the decisions. Payment capture belongs to Payments. Journal posting belongs to Accounting. Enrollment activation belongs to Enrollment.

Chapter 38: Contract Catalog

> **Chapter motto:** A system without a contract catalog makes integration tribal knowledge.

Purpose

A contract catalog records public contracts, owners, versions, consumers, compatibility rules, and deprecation status.

Example

```
contracts:
  PaymentCapturedV1:
    owner: Payments
    type: IntegrationEvent
    consumers:
```

```
      - Accounting
      - Enrollment
      - Reporting
    compatibility: additive_changes_only
    status: active

  JournalPostedV1:
    owner: Accounting
    type: IntegrationEvent
    consumers:
      - Reporting
    compatibility: immutable
    status: active
```

Why it matters

Without a catalog, teams discover contracts by searching code or asking people. That does not scale. A catalog makes integration visible.

Chapter 39: Governance Pipeline in CI/CD

> **Chapter motto:** Architecture should be checked on every change, not after every crisis.

Pipeline stages

A governance pipeline can run after normal build and test steps.

```
restore
build
unit tests
integration tests
architecture tests
metadata validation
contract validation
dependency graph validation
architecture health report
```

Build failure examples

The pipeline should fail when:

- a context references another context's Domain project
- a contract is not versioned
- ownership metadata is missing
- a forbidden dependency appears
- a breaking contract change is not accompanied by a new version

Developer experience

Governance messages must be specific. A vague failure creates frustration. A good message says what rule was violated and how to fix it.

Chapter 40: Architecture Health Score

> **Chapter motto:** A score is not truth; it is a trigger for better questions.

Purpose

An architecture health score summarizes signals that indicate whether the system is becoming harder to change.

Inputs

Possible inputs include:

- ownership violations
- circular dependencies
- cross-context change rate
- contract volatility

- dependency growth
- governance failures
- ADR coverage
- metadata completeness

Example model

```
{
  "architectureHealthScore": 86,
  "ownershipViolations": 0,
  "contractVolatility": "medium",
  "crossContextChangeRate": "18%",
  "recommendations": [
    "Review Reporting dependencies",
    "Stabilize Payments contract consumers"
  ]
}
```

Warning

Do not worship the score. Use it to focus review conversations.

Chapter 41: Repository Template

> **Chapter motto:** A good repository teaches the architecture before a meeting is needed.

Recommended structure

```
src/
  Host/
  Business/
    Identity/
    Accounting/
    Payments/
    Enrollment/
```

```
    Reporting/
  Platform/
  Governance/
  Shared/

tests/
  UnitTests/
  IntegrationTests/
  ArchitectureTests/

documentation/
```

Why structure matters

A repository is a learning surface. New developers and AI agents read it to infer the system. If the structure hides ownership, every change starts with confusion.

Shared warning

Shared must stay small. Shared should not contain business entities, business services, or shared domain models. Shared is for very stable technical abstractions.

Chapter 42: .NET Implementation Reference

> **Chapter motto:** Use frameworks to express architecture, not to replace it.

Suggested stack

A .NET implementation may use ASP.NET Core, EF Core, MediatR, FluentValidation, OpenTelemetry, and architecture tests. These are tools, not the architecture itself.

Context internal structure

```
Identity.Contracts
Identity.Domain
Identity.Application
Identity.Infrastructure
Identity._Docs
```

Dependency direction

Domain should be framework-independent. Application orchestrates use cases. Infrastructure implements persistence and external integrations. Contracts are public.

Important rule

Do not let the framework dictate ownership. ASP.NET controllers are delivery mechanisms. EF Core entities are persistence mechanisms. Business ownership lives in contexts.

Chapter 43: Anti-Patterns

> **Chapter motto:** The most expensive architecture mistakes usually begin as small conveniences.

Shared domain model

A shared domain model looks efficient. It reduces mapping code. Later, every context depends on the same internal object and no one can change it safely.

Common service

A service named `CommonService` often becomes a dumping ground for unowned behavior. ABA prefers clear ownership over shared convenience.

Direct database integration

Reading another context's database may be fast today. It couples persistence and business ownership. Use contracts or purpose-built read models.

Unversioned events

Events without versioning are unstable public APIs. Version them from the beginning.

Platform with business rules

If Platform decides whether a refund is allowed or a journal can be posted, Platform has become a hidden business context.

Chapter 44: Adoption Guide

> **Chapter motto:** Adopt ownership first, tools second.

Start small

Do not start ABA by restructuring the whole repository. Start with the most painful business flow.

Good candidates:

- payment to accounting
- registration with payment
- user membership and authorization
- reporting from operational data

First month plan

Week 1: map ownership and identify violations.

Week 2: introduce metadata files for one or two contexts.

Week 3: add architecture tests for forbidden dependencies.

Week 4: refactor one cross-context flow to use contracts.

Cultural adoption

Developers must understand that ABA is not a punishment. It is a way to avoid future pain. Governance should teach, not merely block.

Chapter 45: Practical Review Checklist

> **Chapter motto:** A checklist is a memory system for architecture.

Review questions

Before approving a significant change, ask:

- Which context owns this capability?
- Which data does it create or modify?
- Which business rule does it enforce?
- Does it introduce a new contract?
- Does a contract version need to change?
- Does governance enforce the new boundary?
- Does metadata need to be updated?
- Does this change increase or reduce future cost of change?

Pull request comment example

Instead of saying, "This is not clean," say:

"This change makes Payments depend on Accounting.Domain. That means future changes to journal internals may break payment behavior. Please publish a Payments contract and let Accounting consume it."

Specific architecture feedback is more useful than abstract criticism.

Chapter 46: For Managers and Technical Leaders

> **Chapter motto:** Architecture quality is a delivery strategy, not an engineering luxury.

Why leaders should care

When architecture degrades, business delivery slows. Teams spend more time coordinating, testing unrelated areas, and recovering from incidents. The cost appears as missed deadlines, unclear ownership, and fear of change.

What to measure

Leaders should not only ask how many features were delivered. They should ask whether the system is becoming easier or harder to change.

Useful signals:

- lead time for change
- number of teams required per feature
- release confidence
- recurring architecture violations
- time spent in coordination meetings

How to support ABA

Support clear ownership. Avoid rewarding shortcuts that create long-term cost. Give teams time to automate governance. Treat ADRs and metadata as part of delivery, not optional paperwork.

Chapter 47: Final Manifesto

> **Chapter motto:** Build systems that can survive the business changing its mind.

The ABA manifesto

Architecture exists to help the business evolve.

A system that works today but cannot change tomorrow is not a success. It is a liability waiting for the next business shift.

ABA stands on ten principles:

1. Optimize for future cost of change.
2. Make ownership obvious.
3. Treat business contexts as authority boundaries.
4. Use contracts as public promises.
5. Keep Platform technical and business-agnostic.
6. Make governance executable.
7. Measure architectural health.
8. Preserve architectural intent.
9. Make architecture readable by AI.
10. Let complexity pay rent.

The goal is simple to say and hard to practice:

Make change safe, cheap, and obvious.

Appendix A: Glossary

Term	Meaning
ABA	Adaptive Business Architecture
Cost of Change	The full cost of safely changing a system
Ownership	Authority over data, rule, decision, or contract
Business Context	A boundary of business ownership
Contract	A public model used across context boundaries
Platform	Reusable technical capability layer
Governance	Executable architecture enforcement
ADR	Architecture Decision Record
Telemetry	Measurement of runtime or architecture health
AI-Navigable Architecture	Architecture that agents can read through metadata
Modular Monolith	One deployable unit with strong internal boundaries
Shared Domain Model	A domain model reused across contexts; an ABA anti-pattern

Appendix B: ownership.yaml Template

```
context: ContextName

data_ownership:
  ResourceName:
    owner: ContextName

rule_ownership:
  BusinessRuleName:
    owner: ContextName
```

```
decision_ownership:
  DecisionName:
    owner: ContextName

contract_ownership:
  ContractNameV1:
    owner: ContextName

forbidden_writers:
  - OtherContext
```

Appendix C: context-map.yaml Template

```
context: Payments

owned_capabilities:
  - Payment lifecycle
  - Refund execution

publishes:
  - PaymentCapturedV1
  - PaymentRefundedV1

consumes:
  - UserCreatedV1

allowed_dependencies:
  - Identity.Contracts
  - Platform.Messaging
  - Platform.Observability

forbidden_dependencies:
  - Accounting.Domain
  - Enrollment.Domain
```

Appendix D: ADR Template

```
# ADR-000: Title

## Status
Proposed / Accepted / Superseded / Retired

## Context
What problem or pressure created the decision?

## Decision
What did we decide?

## Alternatives
What options were considered?

## Tradeoffs
What costs did we accept?

## Consequences
What future impact should teams remember?
```

Appendix E: AI Agent Prompt

```
You are working in an ABA repository.
Before generating code, read documentation, context metadata, ADRs, and AI
rules.
Determine the owning context before making changes.
Do not create cross-context Domain dependencies.
Do not introduce shared domain models.
Use versioned contracts between contexts.
Update metadata when architecture changes.
Run or generate governance tests when boundaries change.
Explain the architectural impact of your change.
```

Appendix F: Publishing Review Checklist

- Is the central idea clear in the first chapter?
- Does each chapter explain why the decision exists?
- Are examples practical and not merely theoretical?
- Are code samples readable?
- Are contracts versioned?
- Are comparisons with DDD, Clean Architecture, and Microservices fair?
- Are repeated paragraphs removed?
- Is the PDF visually readable?
- Can a technical team present the book internally?

Extended English Edition Additions

Chapter 48: Identity Deep Dive: From Login Table to Business Capability

> **Chapter motto:** Login is a feature; identity is an ownership boundary.

Why identity deserves more attention

Many systems begin with a table named `Users`. The table stores an email, password hash, name, and a few flags. This is enough for the first release. Then the business adds organizations, invitations, roles, permissions, delegated

administration, multi-tenant access, external identity providers, single sign-on, and audit requirements. The original table becomes the center of too many responsibilities.

ABA treats Identity as a business capability because the meaning of a user is not purely technical. A user may be active in one organization, suspended in another, invited but not yet accepted, or allowed to manage payments but not accounting journals. These are business facts, not just authentication details.

Traditional design

A traditional layered design often places identity concerns in a technical security folder. Controllers call a user service. The user service checks roles, writes membership data, and sometimes calls payment or enrollment logic. Permission checks appear in controllers, attributes, services, and database queries.

The problem is not that layers exist. The problem is that no boundary explains who owns identity decisions.

ABA design

ABA separates three ideas:

1. Authentication protocol mechanics belong to Platform.Security.
2. User, role, permission, organization, and membership concepts belong to Identity.
3. Business decisions in other contexts consume Identity contracts instead of rewriting identity rules.

For example, Enrollment may need to know whether a participant is allowed to enroll. It should not reimplement role logic. It should ask for the information it needs through a public contract or application API.

Example contracts

```
public sealed record MembershipActivatedV1(
    Guid MembershipId,
    Guid UserId,
```

```
    Guid OrganizationId,
    DateTime ActivatedAt
);

public sealed record RoleAssignedV1(
    Guid UserId,
    Guid OrganizationId,
    string RoleCode,
    DateTime AssignedAt
);
```

Architectural decision

Identity should not become a global utility used by everyone without boundaries. It should publish stable contracts and expose intentional application use cases. Other contexts should never write Identity tables directly.

Review questions

- Is there one owner for user lifecycle?
- Are permissions defined in one place?
- Can another context change membership status directly?
- Can an AI agent discover Identity ownership from metadata?

Chapter 49: Accounting Deep Dive: Chart of Accounts, Journals, and Auditability

> **Chapter motto:** If money moved, the record must outlive the implementation.

Why Accounting is a strong ABA example

Accounting is useful as an example because it makes the cost of bad architecture visible. If financial history can be edited freely, reports become untrustworthy. If payments can create journals directly, financial rules leak into payment code. If reporting tables become the source of truth, audit trails become unreliable.

Chart of Accounts

The Chart of Accounts is not simply a lookup table. It defines how financial activity is classified. Account type matters. Asset, liability, equity, revenue, and expense accounts have different meanings. Changing an account type after transactions exist can corrupt interpretation.

ABA therefore treats account structure as owned by Accounting. Other contexts can reference account codes through contracts or configuration, but they should not own the account lifecycle.

Journal posting

A journal posting is a decision. The system must validate that debit equals credit, the fiscal period is open, the accounts are active, and the journal has enough information to be audited.

Payments can say: payment captured. Accounting decides: what financial entry should represent this event?

Correction strategy

The most important design decision is that corrections should be explicit. A posted journal is not updated. A correction creates a new entry that explains what changed.

This can frustrate users who expect an edit button. ABA does not ignore usability; it changes the UI language. Instead of "edit posted journal," the UI offers "create correction" or "reverse journal." The user gets a workflow that matches business reality.

Implementation notes

A robust Accounting context should include:

- account definitions and account status
- fiscal period lifecycle
- journal validation
- posting rules
- reversal entries
- audit trail
- integration event consumers
- read models for trial balance and ledger views

Governance rules

Governance should prevent commands named like `DeletePostedJournal` or dependencies that allow other contexts to write accounting tables. The accounting boundary must be protected both by design and by automated checks.

Chapter 50: Payments Deep Dive: Providers, Refunds, and Idempotency

> **Chapter motto:** The provider is external; the payment lifecycle is yours.

Provider complexity

Payment providers introduce uncertainty. A callback may arrive late. A callback may arrive twice. A provider may authorize but fail to capture. A refund may be requested, accepted, then fail during settlement. Network calls may time out even when the provider completed the operation.

This is why Payments should own a clear internal lifecycle rather than treating provider responses as direct database updates.

Internal truth vs external response

A provider response is an input. The Payments context decides how that input changes Payment state. This distinction helps prevent provider-specific behavior from spreading through the domain.

Infrastructure adapters can translate Stripe, PayPal, Adyen, or a bank gateway into a common application command. The domain model should not depend on provider SDKs.

Refund ownership

Refund execution belongs to Payments, but refund permission may involve other contexts. For example, Enrollment may decide whether a cancellation is eligible. Accounting may decide how the refund should be posted. Payments owns the act of refunding through the provider.

Idempotency example

```
Provider sends capture callback A.
Payments records capture and publishes PaymentCapturedV1.
Provider sends the same callback again.
Payments recognizes the transaction and does not publish a second event.
```

Failure handling

Payments should record attempts, provider request IDs, provider response IDs, error codes, retry status, and correlation IDs. These are not optional details; they are the difference between a support nightmare and an auditable payment system.

Architecture rule

No business context should call a provider directly except Payments or a specific provider integration owned by Platform/Payments. Otherwise provider logic spreads across the system.

Chapter 51: Enrollment Deep Dive: Capacity, Waitlists, and Policy

> **Chapter motto:** A seat is a business promise, not just a counter.

Why Enrollment is more complex than it looks

Enrollment begins as a simple relationship: a participant joins an offering. Then the business adds capacity, waitlists, approvals, prerequisites, deadlines, payments, cancellations, transfers, and refund rules. A simple join table becomes a workflow.

ABA treats Enrollment as the owner of participation and enrollment policy.

Capacity as a business rule

Capacity is not only a database count. It has meaning. Does a pending enrollment hold a seat? Does a failed payment release a seat? Does an administrator override capacity? Does waitlist order matter? These decisions belong to Enrollment.

Waitlist example

A waitlist should have deterministic promotion rules. If a seat opens, the system should know whether the first waiting participant is promoted automatically, manually, or after payment confirmation.

Interaction with Payments

Payments publishes `PaymentCapturedV1`. Enrollment consumes it. But Enrollment still decides whether activation is allowed. Payment evidence is not the same as enrollment authority.

Interaction with Identity

Identity confirms who the user is and what organization or membership they belong to. Enrollment owns the participation record.

Governance rule

No context except Enrollment should change enrollment state directly. Admin features should call Enrollment application use cases rather than update rows.

Chapter 52: Reporting Deep Dive: Projections Without Ownership Theft

> **Chapter motto:** Read models are allowed to copy facts, not to own them.

Why reporting becomes dangerous

Reporting often starts as harmless dashboards. Then teams need faster queries, so reporting tables are created. Then someone edits the reporting table to fix a dashboard. Then a manager begins trusting the edited dashboard more than the source system. At that moment, Reporting has accidentally become a competing source of truth.

ABA allows reporting projections but forbids ownership theft.

Projection model

Reporting can consume events such as `UserCreatedV1`, `PaymentCapturedV1`, `JournalPostedV1`, and `EnrollmentActivatedV1`. It can build denormalized tables optimized for dashboards.

Correction model

If a value is wrong, correct the owning context. Do not patch Reporting unless the issue is purely projection-related. If Accounting posted a correction, Reporting should consume that correction and update its view.

Performance

Reporting may need separate storage, search indexes, columnar databases, or BI tooling. ABA supports this because these are read-side optimizations. The boundary is crossed only when Reporting begins making business decisions that belong to source contexts.

Review question

Can the Reporting model be rebuilt from source events and owned data? If yes, it is likely a projection. If no, it may have become a hidden source of truth.

Chapter 53: Metadata Deep Dive: Schemas That Teach the Repository

> **Chapter motto:** Metadata is documentation that tools can argue with.

Why plain documents are not enough

A Markdown document can explain architecture, but a governance tool cannot easily enforce a paragraph. Metadata gives the repository a structured architecture vocabulary.

ownership.yaml

This file answers ownership questions. It should be small, explicit, and close to the context.

context-map.yaml

This file explains what the context publishes, consumes, and depends on. It is the starting point for dependency validation and architecture visualization.

ai-rules.yaml

This file tells AI agents what is allowed and forbidden. It should contain direct rules rather than philosophical statements.

```
must_read:
  - ownership.yaml
  - context-map.yaml
  - decisions.md

forbidden:
  - create_cross_context_domain_reference
  - create_unversioned_contract
  - modify_other_context_database
```

contract-catalog.yaml

The contract catalog records public models and consumers. It helps teams understand impact before changing a contract.

Metadata lifecycle

Metadata must change when architecture changes. Stale metadata is worse than missing metadata because it creates false confidence.

Chapter 54: Governance Deep Dive: From Principles to Enforcement

> **Chapter motto:** Architecture should fail fast, before bad structure becomes normal.

Governance levels

ABA governance can start simple and grow over time.

Level 1: manual checklist.

Level 2: architecture tests for dependency rules.

Level 3: metadata validation.

Level 4: contract compatibility checks.

Level 5: custom analyzers and architecture telemetry.

Rule catalog

A governance rule should have an ID, description, severity, reason, and remediation.

```
rules:
  ABA001:
    description: Context Domain projects must not reference other Context
Domain projects.
    severity: error
```

```
    reason: Cross-context domain dependencies break ownership.
    remediation: Depend on the other context's Contracts project instead.
```

Human review still matters

Automation catches structural violations. Humans still review tradeoffs, business meaning, and domain language. The best governance combines both.

Avoid governance theater

If governance creates forms but catches no real problems, it is theater. If it blocks every change with unclear rules, it becomes bureaucracy. Good governance is specific, automated, and teachable.

Chapter 55: AI Agent Deep Dive: A Real Pull Request Scenario

> **Chapter motto:** The agent should explain the boundary it changed, not only the code it wrote.

Scenario

A product owner asks: "When a paid enrollment succeeds, activate the enrollment and show it in reporting."

A naive agent may modify Payments so that it updates Enrollment and Reporting tables directly. The feature appears to work. The architecture is damaged.

ABA agent process

The agent first reads metadata. It discovers that Payments owns payment lifecycle, Enrollment owns enrollment status, Reporting owns read models, and Accounting owns journals.

The agent proposes:

1. Payments publishes `PaymentCapturedV1`.
2. Enrollment consumes `PaymentCapturedV1` and applies policy.
3. Enrollment publishes `EnrollmentActivatedV1`.
4. Reporting consumes `EnrollmentActivatedV1`.
5. Accounting consumes `PaymentCapturedV1` and posts financial truth.

Pull request explanation

A good AI-generated PR should include an architecture impact section:

```
Architecture impact:
- No cross-context Domain dependency added.
- New contract: PaymentCapturedV1, owned by Payments.
- Enrollment consumes the contract through application messaging.
- Reporting projection updated through public event.
- Metadata updated: contract-catalog.yaml and context-map.yaml.
```

This is how AI becomes an architecture participant rather than a code generator only.

Chapter 56: A Practical ABA Workshop

> **Chapter motto:** A team learns architecture by applying it to its own pain.

Workshop goal

The goal of an ABA workshop is not to teach terminology. It is to identify the most expensive change paths in a real system and reduce them.

Agenda

1. List recent changes that were surprisingly expensive.

2. Identify which contexts were involved.
3. Map data ownership.
4. Map decision ownership.
5. Identify direct database access across boundaries.
6. Identify duplicated business rules.
7. Choose one flow to refactor.
8. Define contracts and governance checks.

Output

The workshop should produce:

- ownership map
- context map
- list of boundary violations
- first governance tests
- migration plan for one flow
- ADRs for major decisions

Success condition

A successful workshop ends with one concrete change that can be implemented within two weeks. Architecture improves through applied practice, not only discussion.

Chapter 57: Publishing ABA Inside an Organization

> **Chapter motto:** A book becomes useful when it changes how teams decide.

Internal publishing

An organization can use this book as an internal architecture guide. But a guide is useful only if it becomes part of daily work.

Recommended rollout

Start with a short presentation. Then run a workshop on one real flow. Then introduce metadata for one or two contexts. Then add the first architecture tests.

Avoid the big-bang rollout

Do not announce that every team must rewrite everything according to ABA. That creates resistance. Start with pain, show value, then expand.

Leadership message

The leadership message should be simple: we are not adding architecture for beauty. We are reducing future delivery cost.

Chapter 58: The Final Test for an ABA System

> **Chapter motto:** If change is still mysterious, architecture is not done.

The test

Pick a realistic change request. Ask a developer who did not build the system to answer:

- Which context owns the change?
- Which data is affected?
- Which rule owns the decision?

- Which contracts may change?
- Which tests protect the boundary?
- Which metadata should be updated?

If the developer cannot answer without meetings, the architecture is not obvious enough.

AI version of the test

Ask an AI agent to produce an implementation plan without writing code. If the plan crosses forbidden boundaries, the repository is not AI-navigable enough.

Outcome

Architecture is mature when both humans and tools can navigate it safely.

Chapter 59: Extended Manifesto for Long-Lived Systems

> **Chapter motto:** The best architecture is the one that lets tomorrow's team move without fear.

Long-lived systems are different

A prototype optimizes for learning. A long-lived enterprise system must optimize for change across people, teams, regulations, integrations, and business models.

ABA is designed for the second world.

The promise

An ABA system should make it possible to change one business capability without accidentally rewriting the organization. It should make ownership

explicit enough that new developers can contribute. It should make boundaries enforceable enough that shortcuts do not become structure. It should make intent visible enough that future teams understand why decisions exist.

The responsibility

Architecture is not a one-time design event. It is a continuous responsibility. Every pull request either preserves the future or taxes it.

Chapter 60: Closing Chapter: Make Change Safe, Cheap, and Obvious

> **Chapter motto:** The final product of architecture is confidence.

The final idea

ABA can be summarized in three words: safe, cheap, obvious.

Safe means that change does not silently break business truth. Cheap means that change does not require unnecessary coordination. Obvious means that ownership and boundaries are easy to discover.

What to do next

If you want to apply ABA, do not begin by debating every pattern. Begin by choosing one painful flow. Map ownership. Define contracts. Add one governance test. Record one ADR. Measure whether the next change becomes easier.

That is how architecture becomes real.

Final sentence

A system is not truly well-designed because it works today. It is well-designed when tomorrow's team can change it with confidence.